

DAY 7: SQL CODING CHALLENGE VIEWS (SIMPLE & COMPLEX) AND TRIGGERS

Database: Company database

Question 1 – SIMPLE VIEW: EmployeeBasicView

Scenario:

HR wants a quick view showing each employee's name, department, and salary without accessing full details.

Task:

Create a **simple view** named EmployeeBasicView that displays:

EmpName, DeptID, and Salary from the Employees table.

Then, query the view to verify results.

Expected Output:

EmpName | DeptID | Salary

Question 2 – COMPLEX VIEW: EmployeeDepartmentView

Scenario:

Management needs a detailed view combining employee and department information.

Task:

Create a **complex view** named EmployeeDepartmentView

that joins Employees and Departments to show:

EmpName, DeptName, Location, and Salary.

Expected Output:

EmpName | DeptName | Location | Salary

Question 3 – COMPLEX VIEW with Aggregation

Scenario:

The finance department wants to analyze average salary and employee count per department.

Task:

Create a **complex view** named DeptSalaryStats

that shows each department's name, average salary, and number of employees.

Expected Output:

DeptName | AvgSalary | TotalEmployees

Question 4 – UPDATE Using VIEW

Scenario:

HR wants to increase salary by ₹5,000 for all employees visible in the view.

Task:

Use the EmployeeBasicView to update each employee's salary by ₹5,000.

Ensure the change reflects in the main Employees table.

Expected Output:

EmpName | Updated_Salary

Question 5 – DROP VIEW

Scenario:

After the financial analysis is complete, some views are no longer needed.

Task:

Drop the DeptSalaryStats view from the database.

Expected Output:

Confirmation that the view was removed successfully.

Question 6 – TRIGGER (BEFORE INSERT)

Scenario:

HR wants to prevent inserting an employee with a salary below ₹30,000.

Task:

Create a **BEFORE INSERT trigger** named check_min_salary on the Employees table to block any insert with Salary < 30000.

Expected Output:

Attempt to insert such a record → should return an error message like "Salary must be at least 30000"

Question 7 – TRIGGER (AFTER INSERT – Audit Log)

Scenario:

The admin team wants to keep a record every time a new employee is added.

Task:

Create a new table EmployeeAudit(EmpID, EmpName, Action, ActionDate).

Then create an **AFTER INSERT trigger** named log_employee_insert to insert a record into EmployeeAudit whenever a new employee is added.

Expected Output:

EmployeeAudit table shows log entries:

EmpID | EmpName | Action | ActionDate

Question 8 – TRIGGER (AFTER UPDATE – Salary Change Log)**Scenario:**

Finance wants to track salary changes for compliance.

Task:

Create a table SalaryLog(EmpID, OldSalary, NewSalary, ChangeDate).

Then create an **AFTER UPDATE trigger** named log_salary_change that logs every time an employee's salary is updated.

Expected Output:

SalaryLog shows:

EmpID | OldSalary | NewSalary | ChangeDate

Question 9 – TRIGGER (BEFORE DELETE – Block Action)**Scenario:**

The HR policy restricts deleting employees from the IT department.

Task:

Create a **BEFORE DELETE trigger** named prevent_it_delete that prevents deletion of employees where DeptID corresponds to the IT department.

Expected Output:

When trying to delete such an employee → show an error message like "Cannot delete employees from IT department"

Question 10 – TRIGGER (AFTER DELETE – Archive Record)**Scenario:**

When any employee leaves the company, HR wants their record saved in an archive table.

Task:

Create a new table EmployeeArchive(EmpID, EmpName, DeptID, Salary, ExitDate).

Then create an **AFTER DELETE trigger** named archive_deleted_employee that automatically inserts the deleted record into EmployeeArchive.

Expected Output:

Deleted employee appears in EmployeeArchive with exit timestamp.

